

listing

name=forráskód,

listname=Forráskódjegyzék,



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSI NYELVEK ÉS FORDÍTÓPROGRAMOK TANSZÉK

Transzpilált nyelv koncepcióigazolása fordítási idejű invariánsokkal

Témavezető:

Témavezető Neve

Egyetemi docens

Szerző:

Hallgató Neve

Programtervező informatikus BSc

Budapest, 2026

Contents

Chapter 1

Bevezetés

Manapság az informatikának egyre több szektorában jelenik meg az az igény, hogy a kód ne csak hatékony, hanem bizonyíthatóan helyes is legyen. A beágyazott rendszerek, az orvosi szoftverek, vagy a pénzügyi tranzakciókat végző modulok esetében egyetlen szoftverhiba is katasztrofális következményekkel járhat.

1.1 A problémafelvetés

Gyakran egyensúlyt kell keresnünk a program hatékonysága és a helyesség ellenőrzése között. Akárhányszor egy függvénynek elő- vagy utófeltételt szeretnénk megfogalmazni és azokat kódban ellenőrizni (például *assert* utasításokkal), valamennyi futásidejű teljesítményt fel kell áldoznunk. Számos szituáció előfordul, ahol egy erősebb heurisztika megfogalmazásával hatékonyabb kódot tudnánk írni, de a futásidejű ellenőrzések lassítják a szoftvert.

Ennek a problémának megoldásaként merült fel egy olyan imperatív programozási nyelv létrehozása, mely ezeket az elő- és utófeltételeket, invariánsokat ha lehetséges, akkor fordítási időben (compile-time) ellenőrzi.

1.2 Elméleti háttér

Ahhoz, hogy a fordítási idejű invariáns-ellenőrzést megvalósítsuk, több informatikai és matematikai koncepciót is ötvöznünk kell:

- **Hoare-logika:** A programok helyességének formális bizonyítására szolgáló rendszer, amely előfeltételekből (preconditions), parancsokból és utófeltételekből (postconditions) áll.
- **Szimbolikus végrehajtás (Symbolic Execution):** A program nem konkrét értékekkel (pl. $x = 5$), hanem szimbolikus változókkal fut, így a program minden lehetséges végrehajtási útvonala (path) feltérképezhetővé válik.
- **SMT Megoldók (Z3):** A Satisfiability Modulo Theories (SMT) megoldók képesek bonyolult logikai formulák kielégíthetőségét vizsgálni. Projektünk a Microsoft által fejlesztett Z3 SMT megoldót használja a matematikai és memóriabiztonsági állítások igazolására.

Chapter 2

Felhasználói dokumentáció

A szoftver célközönsége elsősorban azon fejlesztők és kutatók csoportja, akik magas megbízhatóságú szoftvereket terveznek, és szeretnék a kódjukban lévő szerződéseket (contracts) fordítási időben verifikálni[cite: 31, 34].

2.1 Telepítés és Rendszerkövetelmények

A fordítóprogram és az analízátor futtatásához a következő szoftveres környezet szükséges[cite: 35]:

- **GHC (Glasgow Haskell Compiler):** A forráskód lefordításához (ajánlott verzió: 9.x).
- **Cabal vagy Stack:** A Haskell csomagfüggőségek (pl. Megaparsec, Z3) kezeléséhez.
- **Z3 SMT Solver:** A gazdagépen globálisan telepítve kell lennie a Z3 binárisnak és a C API-nak (pl. Ubuntu alatt `apt install z3 libz3-dev`).

Az indítás a `cabal run` paranccsal történik, mely paraméterként várja a bemeneti forrásfájlt[cite: 36].

2.2 A nyelv szintaxisa és szemantikája

A létrehozott nyelv egy erősen típusos, C-szerű imperatív nyelv. Támogatja az alapvető típusokat (`Int`, `Double`, `Bool`, `Char`), valamint a mutatókat (`Ptr`), tömböket

(Array), és a komplex adatszerkezeteket (struct, union, enum).

2.2.1 Függvények és Szerződések (Contracts)

A nyelv legfőbb újítása a függvény-szignatúrákba ágyazott szerződések rendszere. Egy függvény deklarációja az argumentumok után opcionálisan tartalmazhat egy előfeltétel-blokkot és egy utófeltétel-blokkot:

```
1 Int divide(Int a, Int b)(b > 0)(res != 0) {  
2     return a / b;  
3 }
```

A fenti kódban a `(b > 0)` a hívó félre vonatkozó előfeltétel, míg a `(res != 0)` a függvény által garantált utófeltétel (ahol a `res` kulcsszó a visszatérési értéket jelöli).

2.2.2 Ciklus invariánsok

A `while` ciklusok elemzése szimbolikus úton gyakran végtelen állapottérhez vezet. Ezt a nyelv a ciklusinvariánsok explicit megadásával kezeli:

```
1 while (i < 10) (i >= 0) {  
2     i = i + 1;  
3 }
```

2.3 Hibaüzenetek és diagnosztika

Amikor a statikus analízátor egy szerződés megsértését vagy egy memóriahibát (pl. tömb alul/túlindexelés) észlel, a fordítás leáll, és a rendszer a következő formátumú diagnosztikai üzenetet adja[cite: 41]: fájlnev:sor:oszlop - Hiba típusa: Kifejezés

Például, ha a fenti `divide` függvényt nullával hívjuk meg:

```
1 test-input:5:5 - Precondition Failed in: divide: y > 0
```

Chapter 3

Megoldási terv (Rendszerterv)

Ez a fejezet bemutatja az alkalmazás architekturális felépítését, az adatfolyamot és a memóriamodellt[cite: 45].

3.1 Architektúra és Adatfolyam

A rendszer három fő logikai rétegre bontható[cite: 47]:

1. **Lexer és Parser:** Beolvassa a nyers szöveget, ellenőrzi a szintaktikai szabályokat, és felépíti az Absztrakt Szintaxisfát (AST).
2. **Analyzer (Hibrid Állapotgép):** Bejárja az AST-t. Ahol lehetséges, konkrét kiértékelést (Concrete Evaluation) végez, ahol nem (pl. ismeretlen szenzoradatok, függvényparaméterek), ott szimbolikus végrehajtásra tér át.
3. **Z3 SMT Backend:** A szimbolikus formulákat lefordítja a Z3 SMT megoldó nyelvére, és matematikai bizonyításokat végez az invariánsokon és biztonsági szabályokon.

3.2 Az Absztrakt Szintaxisfa (AST) szerkezete

A nyelv szintaxisát a `Parser.hs` modulban definiált ADT-k (Algebraic Data Types) írják le[cite: 50].

```
1 data Expr = Expr SourcePos ExprNode
```


Minden kifejezés (**Expr**) egy **SourcePos** csomagolóval van ellátva. Ez kritikus a pontos hibaüzenetek generálásához az Analizátor fázisban.

A memóriakezelést és a végrehajtás-vezérlést a **StmtNode** írja le, amely magában foglalja a deklarációkat, értékadásokat, és a vezérlési szerkezeteket (**If**, **While**, **For**).

3.3 Memóriamodell és Állapotkezelés

A szimbolikus elemző működéséhez elengedhetetlen egy robusztus memóriamodell[cite: 49]. Az állapotot az **AnalyzerState** rekord fogja össze:

- **SymEnv**: A változóneveket memóriacímekre (**Addr**) képezi le.
- **Heap**: Konkrét futásidejű értékeket (pl. **VInt 5**) tárol az adott memóriacímeken.
- **SymStore**: A szimbolikus végrehajtás során keletkező matematikai kifejezéseket, "képleteket" tárolja.
- **facts**: A végrehajtási útvonal (**Path Condition**) során összegyűjtött logikai állítások listája.

Chapter 4

Megvalósítás

Ebben a fejezetben a funkcionális programozási paradigma sajátosságait, a monadikus transzformátorokat és az algoritmusok részleteit tekintjük át[cite: 56]. A Haskell nyelv választása lehetővé tette a kód erős strukturáltságát és a magas megbízhatóságot[cite: 61].

4.1 A Szintaktikai Elemző (Parser) Implementációja

A parser a Megaparsec könyvtárra épül. A kifejezések elemzésére a `Control.Monad.Combinators.Expr` modul `makeExprParser` függvényét alkalmaztam, amely lehetővé tette egy precedencia-táblázat (`operatorTable`) definiálását:

```
1 operatorTable :: [[Operator Parser Expr]]
2 operatorTable =
3   [ [ prefix "-" (UnOp "-"), prefix "!" (UnOp "!")
4     , prefix "*" Deref, prefix "&" AddrOf ]
5   , [ binary "*" (BinOp "*"), binary "/" (BinOp "/") ]
6   , [ binary "+" (BinOp "+"), binary "-" (BinOp "-") ]
7   ...
```

Ez garantálja a C-nyelvi precedenciaszabályok pontos betartását, elkerülve a terjedelmes és nehezen olvasható rekurzív ereszkedő (recursive descent) kód megírását.

4.2 A Szimbolikus Végrehajtó Motor (symExec)

A rendszer "lelke" a `symExec` algoritmus, amely a szimbolikus kiértékelést végzi. Az elágazások (`If`) kezelése különösen komplex: Amikor a szimbolikus végrehajtó egy elágazáshoz ér, ahol a feltétel értéke fordítási időben nem dönthető el (ismeretlen), a motor **kettéágazik**. Párhuzamosan kiértékeli a *Then* és az *Else* ágakat.

A két ág végén a memóriastruktúrák (`SymStore`) eltérhetnek. Az egyesítést (`Merge`) egy Ternáris operátor generálásával oldottam meg, amely a feltétel függvényében választja ki a megfelelő értéket (SSA - Static Single Assignment formához hasonlóan).

4.3 Biztonsági Ellenőrzések (Safety Extraction)

Minden egyes kifejezés kiértékelése előtt lefut az `extractSafety` függvény, amely implicit előfeltételeket generál[cite: 67]. Például osztás operátor detektálása esetén automatikusan generál egy `b != 0` ellenőrzést, vagy tömb indexelés esetén a Z3-ba elküldi az alsó és felső indexhatárok ellenőrzését:

```
1 (pos, "Array Index Out of Bounds", genExpr (BinOp "<" idx (genExpr  
  ↪ (IntLit sz))))
```

4.4 Z3 Integráció és a proveZ3 függvény

Az SMT megoldóval való kommunikáció a `Z3.Monad` interfészen keresztül történik. A `translateZ3` függvény az AST-t Z3 sortokká (típusokká) és formulákká konvertálja. A bizonyítás trükkje: nem magát a feltételt akarjuk igazolni (mert az SMT a kielégíthetőséget – SAT – vizsgálja), hanem a feltétel **tagadását** (negation) adjuk a megoldónak. Ha a tagadás *Unsat* (kielégíthetetlen), akkor a tétel bizonyítottan minden lehetséges esetben **Igaz**.

Chapter 5

Tesztelés

A szoftver robusztusságának és helyességének bizonyítása érdekében kiterjedt tesztelési keretrendszert építettem ki a `Hspec` modul használatával[cite: 70].

5.1 Parser Modultesztek

A parser tesztelése során feketedoboz tesztelést alkalmaztam. Az elemzett AST fát egy speciális `clearPos` függvény segítségével megtisztítottam a `SourcePos` adatoktól, így lehetővé vált a tiszta strukturális összehasonlítás. Teszteltem a komplex mutatókonfigurációkat (pl. `const Int * const p;`), a precedenciaszabályokat, és a negatív tesztek is (olyan kódok, amiknek szintaktikai hibával el kell hasalniuk).

5.2 Analizátor és Z3 Tesztesetek

Az analizátor tesztelése a legfontosabb, hiszen itt bizonyosodik be a szoftver hozzáadott értéke. A tesztesetek az alábbi kategóriákat fedik le:

5.2.1 Implicit utófeltételek felfedezése

A motor képes a "sandbox" végrehajtás segítségével kitalálni a paraméterek módosulásait és a visszatérési értékek összefüggéseit anélkül, hogy a felhasználó ezt külön annotálta volna:

```
1 Int add(Int a, Int b) { return a + b; }  
2 Void process(Int val, Int x, Int y)(val == x + y) { return; }
```

A fenti kódban a `process` függvény sikeresen lefut, mert a fordító automatikusan generált egy `res == a + b` utófeltételt az `add` függvényhez.

5.2.2 Tömb határellenőrzések (Array Bounds Checking)

A rendszer Z3 segítségével bizonyítani tudja az indexek helyességét. Ha az elemző a `arr[10] = 99;` sort látja egy 5 elemű tömbre, sikeresen dobja a `Array Index Out of Bounds: 10 < 5` fordítási idejű hibát.

5.2.3 Ciklusok feloldása (Bounded Model Checking)

A tesztek bizonyítják, hogy a kód képes automatikusan "kigöngyölni" a ciklusokat (loop unrolling). Egy 3 iterációs ciklust explicit invariánsok megadása nélkül is képes matematikailag bizonyítani a rendszer, amely jelentősen növeli a felhasználói élményt és csökkenti a fejlesztőre háruló teher mértékét.

Chapter 6

Összegzés és Értékelés

A bemutatott projekt keretében sikeresen elkészült egy olyan koncepcióigazoló (Proof of Concept) compiler technológia, amely egyesíti a hagyományos imperatív nyelvek szintaxisát a formális bizonyító rendszerek erejével.

6.1 Szoftver futtatása és Robusztusság

A tesztek és a kézi futtatások alapján a program a tervnek megfelelően működik[cite: 79]. Robusztussága kiemelkedő: hibás bemenet, végtelen ciklust eredményező forráskód, vagy ellentmondásos szimbolikus állítások esetén sem omlik össze ("száll el"). Az erőforrásokkal jól gazdálkodik (a `fuel` alapú limitált szimbolikus fabejárás garantálja, hogy a futásidő nem nő exponenciálisan a végtelenségig)[cite: 81, 83].

6.2 További fejlesztési lehetőségek

A jövőben a nyelv tovább bővíthető dinamikus memóriakezeléssel (pl. `malloc/free`), amelynek verifikálása a Separation Logic (szeparációs logika) beépítését igényelné. Szintén hasznos lenne egy kódgeneráló (Backend) modul írása, amely a megbízhatónak ítélt AST-t C vagy LLVM IR (Intermediate Representation) kódra fordítaná, így a lefordított program ténylegesen futtathatóvá válna operációs rendszer szinten is.